Before start, make sure you've read the following materials:

- Tour of Go: Function values, Function closures, Exercise: Fibonacci closure
- Tour of Go: Defer and Stacking defers
- Tour of Go: Errors
- Go Blog: Defer, Panic, and Recover
- Go Blog: Error handling and Go
- Go Blog: Working with Errors in Go 1.13
- Go Blog: Fixing For Loops in Go 1.22
- Errors in Go: handling, wrapping and best practices
- Error handling in Go: Defer, Panic and Recover
- Error handling in Go (panic, wrapping, defer)
- Go by Example: Closures, Errors, Defer, Panic, Recover

Part 1: Functions and closures

> **Important**
>
> *In Go, functions are **values**. You can store a function in a variable, put it in a map or slice, pass it to another function and return it from a function — just like an `int` or a `string`.*

Functions as values

```
func apply(nums []int, f func(int) int) []int {
    out := make([]int, len(nums))
    for i, n := range nums {
        out[i] = f(n)
    }
    return out
}

func double(n int) int { return n * 2 }

func main() {
    fmt.Println(apply([]int{1, 2, 3}, double))                              // [2 4 6]
    fmt.Println(apply([]int{1, 2, 3}, func(n int) int { return n * n }))   // [1 4 9]

    var f func(string) string = strings.ToUpper
    fmt.Println(f("go")) // GO
}
```

Key Points:

- `func(int) int` is a **type: "a function that takes an** `int` **and returns an** `int` **".**
- `func(n int) int { ... }` **without a name is an anonymous function (a function literal).**
- **The zero value of a function type is** `nil`**. Calling a** `nil` **function panics.**

A function type with a name

```
type Op func(a, b float64) float64

ops := map[string]Op{
    "+": func(a, b float64) float64 { return a + b },
    "*": func(a, b float64) float64 { return a * b },
}
fmt.Println(ops["+"](2, 3), ops["*"](2, 3)) // 5 6
```

Recap: multiple results, named results, variadic functions

```go
func divmod(a, b int) (q, r int) { // named results
    q = a / b
    r = a % b
    return // "naked" return: returns q and r
}

func sum(nums ...int) int { // variadic: nums is a []int
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}

q, r := divmod(17, 5)                          // 3 2
fmt.Println(sum(), sum(1, 2), sum([]int{4, 5, 6}...)) // 0 3 15
```

> **Tip**
>
> **Named results are useful for documentation and, as you'll see in Part 4, for `defer`. Avoid naked `return` in long functions — the reader has to scroll up to see what is returned.**

Closures

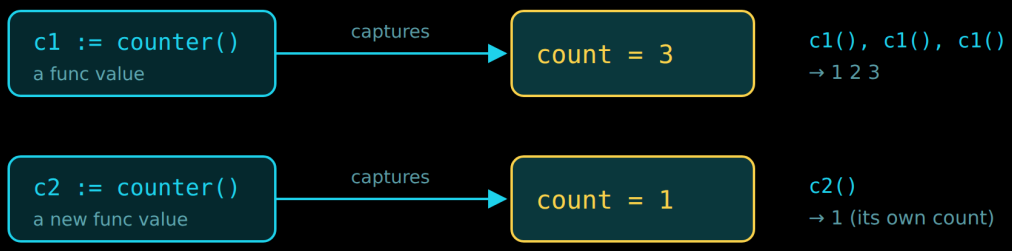
A closure is a function that uses variables declared outside of it. The function "closes over" these variables and keeps them alive.

```go
func counter() func() int {
    count := 0
    return func() int {
        count++
        return count
    }
}

func main() {
    c1 := counter()
    fmt.Println(c1(), c1(), c1()) // ?
    c2 := counter()
    fmt.Println(c2(), c1())       // ?
}
```

A closure = function + the variables it captured

```
func counter() func() int {
    count := 0
    return func() int { count++; return count }
}
```



Each call of `counter()` creates a new `count`.

The variable lives as long as the closure that uses it — even after counter() returned.

Question

Predict both lines. `count` is a local variable of `counter()`, and `counter()` has already returned. Why does `count` still exist?

Closures capture variables, not values

```
x := 10
show := func() { fmt.Println("x =", x) }
x = 20
show() // ?
```

The closure refers to the variable `x` itself, so it sees its current value at the moment it runs.

A closure that builds functions

```
func multiplier(k int) func(int) int {
    return func(n int) int { return n * k }
}

triple := multiplier(3)
fmt.Println(triple(5)) // 15
```

> ⚠ **Caution**
>
> **Closures in loops: the Go version matters!**

```go
var funcs []func()
for i := 0; i < 3; i++ {
    funcs = append(funcs, func() { fmt.Print(i, " ") })
}
for _, f := range funcs {
    f()
}
```

`go` line in `go.mod`	Output	Why
`go 1.21` or older	`3 3 3`	one variable `i` for the whole loop; all closures share it
`go 1.22` or newer	`0 1 2`	every iteration gets its **own** `i`

Old articles and interview questions still show the `3 3 3` version. Know both — and check the `go` line in `go.mod` of the project you work on.

> ✎ **Task**
>
> **Task 1: Closures**
>
> **1. Write `makeAccumulator() func(int) int`. Every call adds the argument to a running total and returns the total: `acc(10), acc(5), acc(-3)` → `10 15 12`.**
>
> **2. Write `memoize(f func(int) int) func(int) int`. It returns a function that remembers results in a map (Week 3). Prove with a counter that the slow function is called only once per input.**
>
> **3. Create a module with `go 1.21` in `go.mod`, run the loop example above, then change it to `go 1.22`. Show both outputs.**
>
> **Participation Task: explain on the defence where the `count` variable of `counter()` lives and why two counters don't share it.**

> ✎ **Task**
>
> **Task 2: Solve the problem from the tour of Go -> Exercise: Fibonacci closure**

Part 2: Errors as values

> **Important**
>
> ***Go has no exceptions and no `try/catch`. An error is an ordinary value that a function returns, usually as its last result. The caller checks it with `if err != nil`.***

Remember Week 5: `error` is just an interface.

```
type error interface {
    Error() string
}
```

Errors from the standard library

```
n, err := strconv.Atoi("42a")
fmt.Println(n, err) // 0 strconv.Atoi: parsing "42a": invalid syntax

_, err = os.Open("missing.txt")
fmt.Println(err)    // open missing.txt: no such file or directory
```

The usual pattern:

```
f, err := os.Open(name)
if err != nil {
    return fmt.Errorf("open config: %w", err) // handle it or return it
}
// use f only after the check
```

Sentinel errors

A sentinel error is a package-level variable that callers can compare against:

```
var ErrNotFound = errors.New("student not found")

func GetGrade(name string) (int, error) {
    g, ok := grades[name]
    if !ok {
        return 0, ErrNotFound
    }
    return g, nil
}
```

The standard library is full of them: `io.EOF`, `fs.ErrNotExist`, `sql.ErrNoRows`, `strconv.ErrSyntax`.

Custom error types

When the caller needs details, create a type that implements `error`:

```
type GradeError struct {
    Student string
    Grade   int
}

func (e *GradeError) Error() string {
    return fmt.Sprintf("invalid grade %d for %s", e.Grade, e.Student)
}

func SetGrade(name string, g int) error {
    if g < 0 || g > 100 {
        return &GradeError{Student: name, Grade: g}
    }
    grades[name] = g
    return nil // a literal nil! (Week 5, nil interfaces)
}
```

Rules of thumb

- Error messages start with a lowercase letter and have no period at the end: they are often glued into longer messages.
- Handle an error once: either log it, or return it — not both.
- Don't throw errors away with `_` unless you can explain why it's safe.
- Return `error` (the interface), not `*GradeError`, and `return nil` when there is no error.

> **Question**
>
> Why does Go return errors as values instead of throwing exceptions? Name one advantage and one disadvantage.

Part 3: Error wrapping

When an error travels up through several functions, each layer can add context — what it was doing when the error happened. `fmt.Errorf` with the `%w` verb wraps the original error:

```
var ErrNotFound = errors.New("student not found")

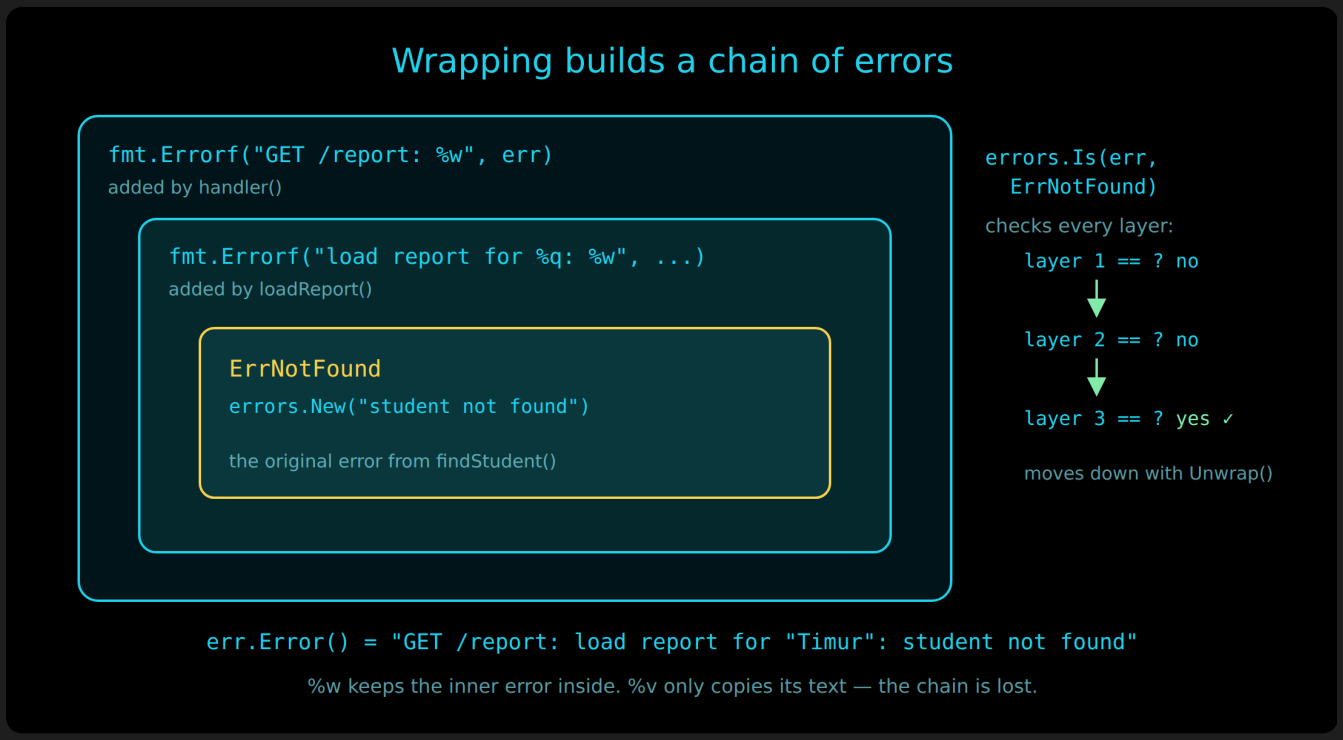
func findStudent(name string) error {
    return ErrNotFound
}

func loadReport(name string) error {
    if err := findStudent(name); err != nil {
        return fmt.Errorf("load report for %q: %w", name, err)
    }
    return nil
}

func handler() error {
    if err := loadReport("Timur"); err != nil {
        return fmt.Errorf("GET /report: %w", err)
    }
    return nil
}

func main() {
    err := handler()
    fmt.Println(err)
    fmt.Println(err == ErrNotFound)          // ?
    fmt.Println(errors.Is(err, ErrNotFound)) // ?
}
```

```
GET /report: load report for "Timur": student not found
```



The three tools

Function	Question it answers	Use instead of
`errors.Is(err, ErrNotFound)`	Is this error — or anything inside it — `ErrNotFound`?	`err == ErrNotFound`
`errors.As(err, &target)`	Is there an error of type `*GradeError` inside? Give it to me.	`err.(*GradeError)`
`errors.Unwrap(err)`	What is one layer deeper?	— (rarely needed)

```go
var wrapped error = fmt.Errorf("saving: %w", &GradeError{"Dias", 120})

var ge *GradeError
if errors.As(wrapped, &ge) {
    fmt.Println("student:", ge.Student, "grade:", ge.Grade) // student: Dias grade: 120
}
```

> ⚠ **Caution**
>
> **`%v` instead of `%w` breaks the chain**

```go
lost := fmt.Errorf("load report: %v", ErrNotFound)
fmt.Println(lost)                          // load report: student not found
fmt.Println(errors.Is(lost, ErrNotFound))  // false
```

The text looks the same, but the original error is gone — only its message was copied.

Your own type can wrap too

Any error type with an `Unwrap() error` method becomes part of the chain:

```
type QueryError struct {
    Query string
    Err   error
}

func (e *QueryError) Error() string { return "query " + e.Query + ": " + e.Err.Error() }
func (e *QueryError) Unwrap() error { return e.Err }

var ErrTimeout = errors.New("timeout")

err := fmt.Errorf("load users: %w", &QueryError{Query: "SELECT * FROM users", Err:
ErrTimeout})
fmt.Println(errors.Is(err, ErrTimeout)) // true
```

Several errors at once: `errors.Join`

```
joined := errors.Join(ErrNotFound, &GradeError{"Ali", -5})
fmt.Println(joined)
// student not found
// invalid grade -5 for Ali

fmt.Println(errors.Is(joined, ErrNotFound)) // true
fmt.Println(errors.Join() == nil)           // true: joining nothing gives nil
```

Since Go 1.20 you can also use `%w` more than once: `fmt.Errorf("two problems: %w; %w", err1, err2)`.

Errors from the standard library work the same way

```
_, err := os.Open("missing.txt")
err = fmt.Errorf("config: %w", err)

fmt.Println(errors.Is(err, fs.ErrNotExist)) // true

var pe *fs.PathError
if errors.As(err, &pe) {
    fmt.Println("op:", pe.Op, "path:", pe.Path) // op: open path: missing.txt
}
```

Tip

Sentinel or custom type?

Sentinel (`ErrNotFound`) — the caller only needs to know **what** happened → check with `errors.Is` .

Custom type (`*GradeError`) — the caller needs **details** (which student, which value) → get it with `errors.As` .

Task

Task 3: Grade book

1. Create `GradeBook` with methods `Get(name) (int, error)` (returns `ErrNotFound`) and `Set(name, grade) error` (returns `*GradeError` for grades outside 0–100).

2. Write a service function `AddBonus(gb, name, bonus) error` that calls `Get` and `Set` and **wraps** their errors with context.

3. Write `report(err error)` that prints `OK` , `404: ...` (not found), `400: ...` (invalid grade, also print the student and grade from the error) or `500: ...` (anything else). Use `errors.Is` and `errors.As` .

4. Replace one `%w` with `%v` . Which branch of `report` now fires? Explain.

Task

Task 4: Parse all the lines

You get student records as text, one `name, grade` per line:

```
input := `Aruzhan, 95
Dias, 80
Timur
Bota, 99999999999999999999
Ali, 70`
```

Requirements:

- Write `ParseAll(input string) ([]Record, error)` . A bad line must **not** stop the parsing: collect every error and return them together with `errors.Join` .
- Each error must say which line failed (`line 2: ...`) and keep the original error inside (`%w`).
- Missing comma → your own sentinel `ErrBadFormat` . Bad number → the error from `strconv.Atoi` .
- In `main` , check with `errors.Is` whether the result contains `ErrBadFormat` , `strconv.ErrSyntax` and `strconv.ErrRange` , and use `errors.As` to get a `*strconv.NumError` .
- What does `ParseAll` return as the error when all lines are correct?

Part 4: defer, panic and recover

defer

`defer` schedules a function call to run when the surrounding function returns — no matter how it returns. It's Go's way to always clean up: close files, unlock mutexes, finish timers.

```
f, err := os.Open("grades.csv")
if err != nil {
    return err
}
defer f.Close() // written right next to Open, runs at the end
```

Rule 1: deferred calls run in LIFO order (last in — first out)

```
func main() {
    defer fmt.Println("1: deferred first")
    defer fmt.Println("2: deferred second")
    defer fmt.Println("3: deferred third")
    fmt.Println("main body")
}
```

> **Question**
>
> **In which order are the four lines printed?**

Rule 2: arguments are evaluated immediately, at the `defer` line

```
x := 1
defer fmt.Println("deferred x =", x)
defer func() { fmt.Println("closure x =", x) }()
x = 2
fmt.Println("current x =", x)
```

> **Question**
>
> **Predict the three lines. Why do the two deferred calls print different values of `x`?**

Rule 3: a deferred closure can change named results

```
func double() (result int) {
    defer func() { result *= 2 }()
    return 21
}

func notDouble() int {
    result := 21
    defer func() { result *= 2 }()
    return result
}

fmt.Println(double(), notDouble()) // ?
```

This is how real code reports an error from `Close()` that happens after the main work:

```
func save(name string) (err error) {
    f, err := os.Create(name)
    if err != nil {
        return err
    }
    defer func() {
        if cerr := f.Close(); cerr != nil && err == nil {
            err = cerr // don't lose the Close error
        }
    }()
    _, err = f.WriteString("Aruzhan,95\n")
    return err
}
```

Question

What does this program print? Hint: Rule 2.

```
func trace(name string) func() {
    fmt.Println("enter", name)
    return func() { fmt.Println("exit", name) }
}

func work() {
    defer trace("work")()
    fmt.Println("working...")
}
```

> ⚠ **Caution**
>
> **`defer` runs at the end of the function, not at the end of a loop iteration**

```
for _, name := range names {
    f, err := os.Open(name)
    if err != nil {
        return err
    }
    defer f.Close() // all files stay open until the function returns!
    // ... use f
}
```

With 10 000 files you run out of file descriptors. Fix: move the body of the loop into its own function, so each `defer` runs after each file.

---

panic

A panic stops the normal flow of the current goroutine. The function stops immediately, its deferred calls run, then the same happens in the caller, and so on up the stack. If nobody recovers, the program crashes and prints a stack trace.

Runtime panics you have already met:

```
s := []int{1, 2, 3}
fmt.Println(s[5])   // panic: runtime error: index out of range [5] with length 3

var m map[string]int
m["x"] = 1          // panic: assignment to entry in nil map

var p *Student
fmt.Println(p.Name) // panic: runtime error: invalid memory address or nil pointer
dereference

a, b := 1, 0
fmt.Println(a / b)  // panic: runtime error: integer divide by zero
```

You can also panic yourself:

```
func mustPositive(n int) int {
    if n <= 0 {
        panic(fmt.Sprintf("mustPositive: got %d", n))
    }
    return n
}
```

> **Important**
>
> **Error or panic?**

Return an error when…	panic when…
the problem can happen in normal work: bad input, missing file, network down	it's a **bug** in the program: "this can never happen"
the caller can do something about it	the program can't continue in a sane state
almost always	rarely: e.g. `regexp.MustCompile` with a constant pattern at start-up

---

recover

`recover()` stops a panic and returns the value that was passed to `panic`. It only works inside a deferred function.

```
func safeDivide(a, b int) (result int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = fmt.Errorf("recovered: %v", r)
        }
    }()
    return a / b, nil
}

fmt.Println(safeDivide(10, 2)) // 5 <nil>
fmt.Println(safeDivide(1, 0))  // 0 recovered: runtime error: integer divide by zero
```

Notice the trick: `recover` turns a panic into an ordinary error through a named result.

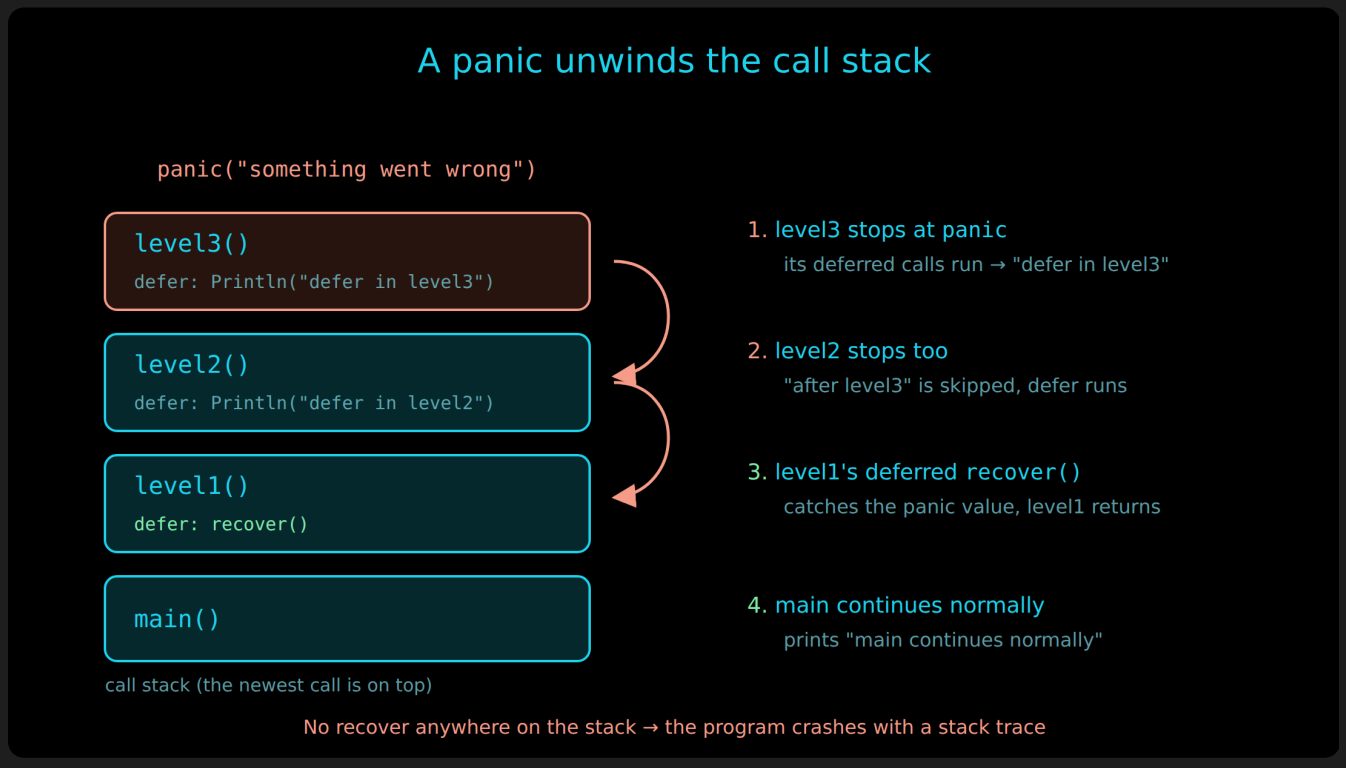
How a panic travels

```go
func level3() {
    defer fmt.Println("defer in level3")
    panic("something went wrong deep inside")
}

func level2() {
    defer fmt.Println("defer in level2")
    level3()
    fmt.Println("after level3") // never runs
}

func level1() {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("recovered in level1:", r)
        }
    }()
    level2()
    fmt.Println("after level2") // never runs
}

func main() {
    level1()
    fmt.Println("main continues normally")
}
```



Question

Only one of the functions `a` and `b` recovers. Which one, and why? What does `c` print?

```go
func helper() {
    if r := recover(); r != nil {
        fmt.Println("recovered:", r)
    }
}

func a() {
    defer func() {
        helper()
    }()
    panic("boom A")
}

func b() {
    defer helper()
    panic("boom B")
}

func c() {
    fmt.Println("c: recover outside defer returned", recover())
}
```

Note

`recover` only catches panics of its own goroutine. A panic in another goroutine crashes the whole program — you'll see this in the concurrency weeks.

Task

Task 5: defer in practice

1. Run the four "predict" snippets of this part and compare the output with your predictions.

2. Write a fake `Open(name) (*Resource, error)` and `(*Resource).Close()` that print `open` / `close`. Write `processAllBad(names)` with `defer` inside the loop and `processAllGood(names)` that fixes it with a helper function. Show the difference in the order of `open` / `close`.

3. Use the `trace` trick to print how long a function took: `defer timer("sort")()`, with `time.Since`.

Task

Task 6: Safe task runner

Write `SafeRun(tasks []Task) []error`, where `type Task struct { Name string; Run func() error }`.

- Every task runs, even if the previous one panicked.
- A returned error becomes `task <name>: <error>` (wrapped with `%w`).
- A panic becomes `task <name>: panic: <value>`.

Test it with tasks that: succeed; index out of range; write to a nil map; call `panic("config is missing")`; return a sentinel error. Then check with `errors.Is` that the sentinel can still be found inside `errors.Join(errs...)`.

Task

Task 7: Benchmark — how expensive are errors, panics and defer?

```go
var ErrNegative = errors.New("negative value")

//go:noinline
func checkErr(n int) (int, error) {
    if n < 0 {
        return 0, ErrNegative
    }
    return n, nil
}

//go:noinline
func checkPanic(n int) int {
    if n < 0 {
        panic(ErrNegative)
    }
    return n
}

func safeCheck(n int) (v int, err error) {
    defer func() {
        if r := recover(); r != nil {
            err = r.(error)
        }
    }()
    return checkPanic(n), nil
}

type Counter struct {
    mu sync.Mutex
    n  int
}

func (c *Counter) IncNoDefer() { c.mu.Lock(); c.n++; c.mu.Unlock() }
func (c *Counter) IncDefer()   { c.mu.Lock(); defer c.mu.Unlock(); c.n++ }
```

Requirements:

- Write `BenchmarkErrorReturn` (`checkErr(-1)`) and `BenchmarkPanicRecover` (`safeCheck(-1)`). How many times slower is the panic path?
- Write `BenchmarkIncNoDefer` and `BenchmarkIncDefer`. Is `defer` expensive?
- Conclude: why should expected failures be **errors**, not panics? Is there a reason to avoid `defer mu.Unlock()`?

⭐ Bonus question (for those who want more)

```go
func main() {
    defer fmt.Println("bye!")
    fmt.Println("hello")
    os.Exit(3)
}
```

What is printed? Why? What does this mean for `log.Fatal`, which calls `os.Exit(1)`?

Theoretical questions for the defence (in-person with Bakytzhan-agai):

- What does "functions are first-class values" mean? Give two examples.
- What is a closure? Where does a captured variable live after the outer function returns?
- What changed for loop variables in Go 1.22?
- Why does Go use error values instead of exceptions?
- What is a sentinel error? When do you need a custom error type instead?
- What is the difference between `%w` and `%v` in `fmt.Errorf`?
- What is the difference between `errors.Is` and `errors.As`?
- In which order do deferred calls run? When are their arguments evaluated?
- How can a deferred function change a function's result?
- When is it OK to panic? Where must `recover` be called to work?

Key takeaways:

- **Functions are values: pass them, return them, store them.**
- **A closure keeps the variables it uses alive. Each call of the outer function creates new ones.**
- **Since Go 1.22, each loop iteration has its own loop variable.**
- **Errors are ordinary values. Return them last, check them with `if err != nil`.**
- **Add context with `fmt.Errorf("...: %w", err)`. `%v` loses the original error.**
- **Check errors with `errors.Is` (which error?) and `errors.As` (which type?), not with `==` or type assertions.**
- **`defer` runs at function exit, in LIFO order; its arguments are evaluated immediately.**
- **Panic for bugs, return errors for everything else. `recover` works only inside a deferred function.**

Defence will be conducted in the next practice lesson, according to your schedule. In-person with your laptops.
